

Reproduction Write-Up

Auto-Encoding Variational Bayes on MNIST

Based on Kingma and Welling, *Auto-Encoding Variational Bayes*

Dataset: MNIST Latent space: 20 dimensions Hidden units: 500

Reproduction date: October 3, 2026

Start Here

- **Goal.** Reproduce the paper’s MNIST variational auto-encoder (VAE) with a 20-dimensional latent space, including the evidence lower bound (ELBO), Gaussian reparameterization, stochastic optimization, reconstruction, prior sampling, t-SNE inspection, and latent interpolation.
- **Allowed software.** The model is implemented in PyTorch; `torchvision` is used only for the MNIST loader and image grids. The optimizer and automatic differentiation are provided by PyTorch.
- **Reproducibility.** The run fixes seed 42, stores all hyperparameters and metrics, and generates every figure in this write-up directly from the final checkpoint.
- **Scope.** This is a focused reproduction, not a claim of bit-for-bit equivalence. The paper’s wake-sleep and Monte Carlo EM baselines are not repeated. All departures are listed in Section [4.3](#).

Deliverables

- `vae_mnist_reproduction_writeup.pdf`: this report.
- `vae_mnist_training/`: runnable multi-file training project with documented class and function names.
- `vae_mnist_training.zip`: portable archive containing source, smoke test, trained checkpoint, configuration, metrics, history, and figures.
- `metrics.json` and `history.csv`: top-level copies of the final scalar results and per-epoch history.

Reproduction Objectives

After completing the experiment, the reader should be able to:

- explain why direct maximum-likelihood learning is difficult when $p_{\theta}(\mathbf{x}) = \int p_{\theta}(\mathbf{x}, \mathbf{z}) d\mathbf{z}$ is intractable;
- derive the ELBO and identify its reconstruction and KL-divergence terms;
- explain why $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ permits low-variance pathwise gradients;
- map each mathematical object to its PyTorch implementation and tensor shape;
- reproduce the reported metrics and visual diagnostics from a clean environment.

Checklist

- ☐ Install Python, PyTorch, torchvision, NumPy, and Matplotlib.
- ☐ Install packages and run `train.py` with the command in Appendix A.
- ☐ Confirm that MNIST contains 60,000 training and 10,000 test images.
- ☐ Confirm that both posterior heads output shape $N \times 20$.
- ☐ Confirm that reconstruction and KL losses are finite and the KL term is non-negative.
- ☐ Confirm that train and test negative ELBO decrease without a widening generalization gap.
- ☐ Inspect reconstructions, prior samples, the t-SNE embedding, and latent interpolations.
- ☐ Compare the final metrics with Table 5.

Reading note. The paper maximizes the ELBO. The implementation minimizes its negative. Therefore, a lower reported “negative ELBO” is better.

Contents

1	Introduction	4
2	Problem Setup and Notation	4
3	Variational Objective	5
3.1	Evidence lower bound	5
3.2	Reparameterization trick	5
3.3	Closed-form KL and implemented loss	5
4	Reproduction Design	6
4.1	Network architecture	6
4.2	Dataset and optimization	6
4.3	Fidelity to the original paper	7
5	Implementation	7
5.1	Training files and function interfaces	7
5.2	Training algorithm	10
5.3	Reproducibility controls	10
6	Results	10
6.1	Quantitative results	10
6.2	Reconstruction	12
6.3	Latent representation	12
6.4	Generative behavior	14
7	Validation and Interpretation	15
7.1	Sanity checks	15
7.2	What the experiment demonstrates	16
8	Conclusion and Limitations	16
A	How to Run the Reproduction	16
B	Loss-to-Code Map	17
C	References	17

1 Introduction

Kingma and Welling propose the stochastic gradient variational Bayes (SGVB) estimator and the auto-encoding variational Bayes (AEVB) algorithm for directed latent-variable models with intractable posteriors [1]. The key idea is to learn two networks jointly:

- a **probabilistic encoder** $q_\phi(\mathbf{z} \mid \mathbf{x})$ that approximates the posterior over latent variables; and
- a **probabilistic decoder** $p_\theta(\mathbf{x} \mid \mathbf{z})$ that models how observations are generated from a latent code.

Unlike a deterministic auto-encoder, the VAE is a normalized generative model. The latent prior constrains the code space, while a likelihood model supplies a statistically meaningful reconstruction term. Reparameterization moves the sampling noise into an auxiliary variable independent of ϕ , allowing gradients to pass through the sample.

MNIST is a suitable reproduction target because the paper uses it directly and its pixels can be modeled with Bernoulli likelihoods. A preliminary two-dimensional run was easy to visualize but produced blurry reconstructions and limited sample diversity. The primary experiment therefore uses $J = 20$, one of the latent sizes studied in the paper. Its geometry is inspected with t-SNE and controlled interpolations rather than forcing the generative model itself to remain two-dimensional. The experiment asks three concrete questions:

1. Does stochastic optimization improve the held-out ELBO?
2. Does the learned posterior reconstruct previously unseen digits?
3. Does sampling and moving continuously through the prior produce coherent digit-like images?

2 Problem Setup and Notation

Let $\mathbf{x} \in [0, 1]^{784}$ be a flattened 28×28 MNIST image and $\mathbf{z} \in \mathbb{R}^J$ a continuous latent vector. The model factorizes as

$$p_\theta(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})p_\theta(\mathbf{x} \mid \mathbf{z}), \quad p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (1)$$

The true posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$ is intractable because evaluating it requires the marginal likelihood

$$p_\theta(\mathbf{x}) = \int p(\mathbf{z})p_\theta(\mathbf{x} \mid \mathbf{z}) d\mathbf{z}. \quad (2)$$

We therefore introduce a diagonal Gaussian approximation:

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\left(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}\left(\boldsymbol{\sigma}_\phi^2(\mathbf{x})\right)\right). \quad (3)$$

Table 1: Notation used throughout the reproduction.

Symbol	Code name	Shape	Meaning
N	batch size	scalar	Number of images in a minibatch; $N = 100$.
$\mathbf{x}$	<code>x</code>	$N \times 784$	Flattened input images.
$\mathbf{h}_e$	encoder hidden	$N \times 500$	Tanh encoder representation.
$\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2$	<code>mu, logvar</code>	$N \times J$	Approximate posterior parameters.
$\boldsymbol{\epsilon}$	noise	$N \times J$	Standard-normal auxiliary noise.
$\mathbf{z}$	latent code	$N \times J$	Reparameterized posterior sample.
$\mathbf{a}$	<code>logits</code>	$N \times 784$	Bernoulli logits produced by the decoder.
ϕ	encoder parameters	–	Weights of encoder, mean head, and log-variance head.
θ	decoder parameters	–	Weights of decoder hidden and output layers.

3 Variational Objective

3.1 Evidence lower bound

Insert $q_\phi(\mathbf{z} \mid \mathbf{x})$ into the marginal log-likelihood and apply Jensen’s inequality:

$$\log p_\theta(\mathbf{x}) = \log \int q_\phi(\mathbf{z} \mid \mathbf{x}) \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} \mid \mathbf{x})} d\mathbf{z} \quad (4)$$

$$\geq \mathbb{E}_{q_\phi(\mathbf{z} \mid \mathbf{x})} [\log p_\theta(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z} \mid \mathbf{x})] \quad (5)$$

$$= \mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x} \mid \mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})) \equiv \mathcal{L}(\theta, \phi; \mathbf{x}). \quad (6)$$

The first term rewards accurate decoding. The KL term regularizes each approximate posterior toward the standard-normal prior, making prior sampling meaningful.

3.2 Reparameterization trick

Sampling $\mathbf{z} \sim q_\phi(\mathbf{z} \mid \mathbf{x})$ naively places a stochastic node between the loss and ϕ . For a Gaussian posterior, write

$$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad \mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \exp\left(\frac{1}{2} \log \boldsymbol{\sigma}_\phi^2(\mathbf{x})\right) \odot \boldsymbol{\epsilon}. \quad (7)$$

The random variable $\boldsymbol{\epsilon}$ is independent of ϕ . Once sampled, the right-hand side is an ordinary differentiable computation, so automatic differentiation yields a pathwise gradient. As in the paper, the reproduction uses one latent sample per datapoint ($L = 1$).

3.3 Closed-form KL and implemented loss

For the diagonal Gaussian posterior and standard-normal prior,

$$D_{\text{KL}}(q_\phi \parallel p) = -\frac{1}{2} \sum_{j=1}^J \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right). \quad (8)$$

MNIST pixels are modeled with independent Bernoulli variables. If a_d is a decoder logit, the per-image negative ELBO minimized in code is

$$\mathcal{J}(\mathbf{x}) = - \underbrace{\sum_{d=1}^{784} [x_d \log \sigma(a_d) + (1 - x_d) \log(1 - \sigma(a_d))]}_{\text{binary cross-entropy / negative log-likelihood}} + \underbrace{D_{\text{KL}}(q_\phi \| p)}_{\text{latent regularizer}}. \quad (9)$$

For numerical stability, the implementation passes logits directly to `binary_cross_entropy_with_logits`.

4 Reproduction Design

4.1 Network architecture

The architecture follows the paper’s one-hidden-layer MLP example. Both hidden layers use tanh, and all linear weights are initialized from $\mathcal{N}(0, 0.01^2)$.

Table 2: Model architecture for the final $J = 20$ experiment.

Stage	Operation	Output shape	Role
Input	Flatten 28×28 image	$N \times 784$	Observation vector.
Encoder	$\tanh(\mathbf{x}W_e^\top + \mathbf{b}_e)$	$N \times 500$	Shared inference features.
Posterior heads	Linear(500, 20) twice	$N \times 20$ each	Produce $\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$.
Sampling	Equation (7)	$N \times 20$	Differentiable stochastic code.
Decoder	$\tanh(\mathbf{z}W_d^\top + \mathbf{b}_d)$	$N \times 500$	Generative hidden representation.
Output	Linear(500, 784)	$N \times 784$	Bernoulli logits.

4.2 Dataset and optimization

Table 3: Final experimental configuration.

Setting	Value
Dataset	MNIST: 60,000 training and 10,000 test images
Input treatment	Scale to $[0, 1]$; dynamic Bernoulli binarization during training
Posterior / prior	Diagonal Gaussian / standard normal
Latent dimension	20
Hidden units	500 in encoder and 500 in decoder
Minibatch / samples	100 images / one $\mathbf{z}$ sample per image
Optimizer	Adagrad, learning rate 0.01
Epochs	50 (30,000 parameter updates)
Initialization	Independent $\mathcal{N}(0, 0.01^2)$ weights, zero biases
Random seed	42
Evaluation	One posterior sample; unbinarized $[0, 1]$ test targets
Device	Apple Metal Performance Shaders (MPS)

4.3 Fidelity to the original paper

Table 4: What is reproduced and what is intentionally simplified.

Component	Paper	This reproduction
Core estimator	AEVB/SGVB with analytic Gaussian KL and $L = 1$	Same
MNIST model	Bernoulli decoder, Gaussian encoder, one 500-unit hidden layer	Same
Batch / optimizer	Batch 100; Adagrad selected from $\{0.01, 0.02, 0.1\}$	Batch 100; fixed Adagrad 0.01
Latent dimensions	Multiple dimensions for bound comparisons, including 20; 2-D used for visualization	One 20-D primary model for reconstruction and generation quality; t-SNE/interpolation for visualization
Training budget	Reported by evaluated examples, extending to many millions	50 epochs = 3.0 million evaluated training examples
Baselines	Wake-sleep and, in one setting, Monte Carlo EM	Omitted
Weight decay	Small MAP-style weight decay, coefficient unspecified	Omitted rather than inventing a coefficient
Evaluation	Variational bound and separate marginal-likelihood study	Negative ELBO plus visual diagnostics

5 Implementation

5.1 Training files and function interfaces

The deliverable is a runnable project rather than a notebook. The map below gives the actual file name, the callable signature to look for, and the returned value. Bodies not essential to the interface are abbreviated with comments; the complete implementations are in `vae_mnist_training/`.

Listing 1: Configuration, data, model, and loss interfaces.

```
# vae_mnist/config.py
@dataclass
class TrainConfig:
    data_dir: str = "work/data"
    output_dir: str = "work/experiment_z20"
    epochs: int = 50
    batch_size: int = 100
    hidden_dim: int = 500
    latent_dim: int = 20
    learning_rate: float = 0.01
    optimizer: str = "adagrad"
    seed: int = 42
    dynamic_binarization: bool = True

# vae_mnist/data.py
def build_mnist_loaders(config: TrainConfig):
    # Download MNIST and create seeded loaders.
    return train_loader, test_loader
```

Listing 2: Model and objective interfaces.

```
# vae_mnist/model.py
class VariationalAutoencoder(nn.Module):
    def reset_parameters(self) -> None:
        # Initializes linear weights from N(0, 0.01^2).

    def encode(self, x):
        return mu, logvar

    @staticmethod
    def reparameterize(mu, logvar):
        std = torch.exp(0.5 * logvar)
        return mu + std * torch.randn_like(std)

    def decode_logits(self, z):
        return logits

    def forward(self, x):
        # Compose encode, reparameterize, and decode_logits.
        return logits, mu, logvar

# vae_mnist/losses.py
def negative_elbo(logits, target, mu, logvar):
    # All three tensors are sums over the current minibatch.
    return nelbo, reconstruction, kl
```

Listing 3: Optimization, utility, and entry-point interfaces.

```
# vae_mnist/engine.py
def run_epoch(model, loader, device, optimizer=None,
              dynamic_binarization=False):
    # optimizer=None selects evaluation mode.
    return {"nelbo": nelbo_per_image,
            "reconstruction": recon_per_image,
            "kl": kl_per_image}

def fit(model, train_loader, test_loader, optimizer,
        device, config):
    # One dictionary per epoch, with train/test ELBO terms.
    return history

# vae_mnist/utils.py
def seed_everything(seed: int) -> None:
    # Seeds Python, NumPy, PyTorch, and MPS.

def choose_device():
    # Selects CUDA first, then MPS, then CPU.
    return device

def save_json(value, path: Path) -> None:
    # Writes a reproducible metadata file.
```

Listing 4: Checkpoint and training entry-point interfaces.

```
# vae_mnist/checkpoint.py
def save_checkpoint(path, model, optimizer, config, metrics):
    # Writes model/optimizer states, config, and metrics.
    return checkpoint_path

def load_checkpoint(path, device):
```

```

# Restores weights and switches the model to eval mode.
return model, config, checkpoint

# train.py
def build_argument_parser():
    return parser

def config_from_args(args):
    return TrainConfig(...)

def build_optimizer(model, config):
    return optimizer # Adagrad by default; Adam is optional.

def main() -> None:
    # Calls fit(), then save_checkpoint(..., "checkpoint.pt").

```

Listing 5: Checkpoint inference used to generate Figures 2, 4, and 5.

```

# vae_mnist/inference.py
def encode(model, images, device):
    return mu, logvar

def reconstruct(model, images, device):
    # Decode posterior means; shape is [N, 1, 28, 28].
    return reconstructions

def interpolate(model, start_images,
               end_images, device, steps=10):
    # Shape is [number_of_pairs, steps, 1, 28, 28].
    return decoded_paths

def generate(model, num_samples, device):
    # Draw  $z \sim N(0, I)$ ; shape is [num_samples, 1, 28, 28].
    return generated_images

# vae_mnist/visualization.py
def save_reconstructions(model, test_loader, device, figure_dir):
    return None # Writes reconstructions.png (Figure 2).

def save_latent_interpolations(model, test_loader,
                              device, figure_dir):
    return None # Writes latent_interpolations.png (Figure 4).

def save_prior_samples(model, device, figure_dir):
    return None # Writes prior_samples.png (Figure 5).

# infer.py
def main() -> None:
    model, config, checkpoint = load_checkpoint(path, device)
    # --task reconstruct, interpolate, generate, or all
    return None

# tests/test_smoke.py
class VaeSmokeTest(unittest.TestCase):
    def test_checkpoint_and_inference_interfaces(self) -> None:
        # Checks checkpoint round-trip and every inference shape.

```

The `pretrained/` directory contains `checkpoint.pt`, the saved configuration, metrics, history, and generated figures. A training run starts at `train.py::main`. General inference starts at `infer.py::main`;

`-task all` regenerates Figures 2, 4, and 5 from the checkpoint without optimization. The broader `render_checkpoint.py::main` entry point additionally recreates the training curves and t-SNE figure.

5.2 Training algorithm

The stochastic training loop is the minibatch AEVB algorithm specialized to MNIST:

1. Sample a shuffled minibatch of 100 images and dynamically binarize its pixels.
2. Encode each image to $(\mu, \log \sigma^2)$.
3. Draw one $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ per image and construct $\mathbf{z}$.
4. Decode $\mathbf{z}$ into 784 Bernoulli logits.
5. Sum binary cross-entropy and analytic KL over the batch, divide by batch size, and backpropagate.
6. Update θ and ϕ jointly with Adagrad.

Dynamic binarization resamples each training pixel as $\text{Bernoulli}(x_d)$ on every visit. This matches the discrete Bernoulli observation model while using the original grayscale intensity as the sampling probability. Test targets remain in $[0, 1]$ to make the held-out curve less noisy.

5.3 Reproducibility controls

The `train.py` entry point seeds Python, NumPy, and PyTorch; uses a seeded data-loader generator; and calls `save_checkpoint` to store the model state, optimizer state, configuration, and metrics. The matching `load_checkpoint` function reconstructs the model in evaluation mode. The run is self-contained: MNIST is downloaded automatically if absent. The separate `infer.py` and `render_checkpoint.py` entry points recreate outputs without repeating training.

6 Results

6.1 Quantitative results

Table 5: Held-out comparison after 50 epochs. Lower negative ELBO and bits/dimension are better.

Metric	Preliminary $J = 2$	Final $J = 20$	Change
Test negative ELBO	166.065	112.599	−53.466 nats/image
Test reconstruction term	161.175	87.900	Sharper reconstructions
Test KL term	4.891	24.698	More encoded information
Bits/dimension	0.3056	0.2072	32.2% reduction
Training examples evaluated	3,000,000	3,000,000	Equal data budget

The 20-D model improves the primary metric by 53.466 nats/image relative to the preliminary 2-D model under the same epoch and minibatch budget. Figure 1 shows that its test negative ELBO falls from 171.4 after the first epoch to 112.6 at epoch 50. The final train–test gap is only 2.34 nats/image.

The KL contribution grows to 24.7 nats/image instead of collapsing, confirming that the additional latent coordinates carry useful information.

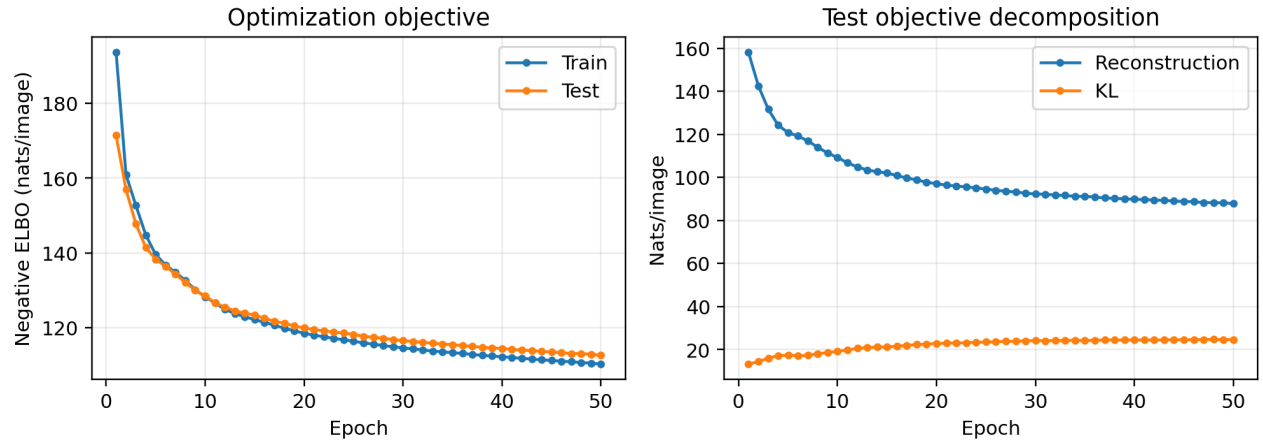


Figure 1: Optimization history and decomposition of the held-out negative ELBO.

6.2 Reconstruction

For reconstruction, the test image is encoded and the posterior mean $\mu(\mathbf{x})$ is decoded. Using the mean removes sampling noise and tests whether the learned representation retains digit identity. Figure 2 shows sharp posterior-mean reconstructions across all twelve examples. Compared with $J = 2$, thin strokes, loops, and digit-specific geometry are substantially better preserved. It is generated directly by `infer.py -task reconstruct`, which calls `load_checkpoint`, `reconstruct`, and `save_reconstructions`.

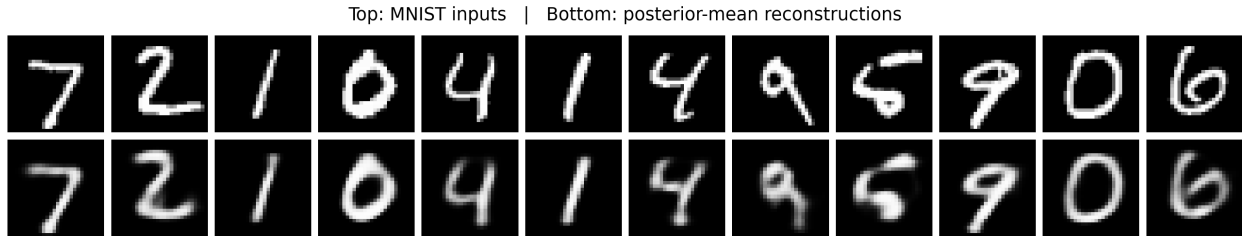


Figure 2: Top row: unseen MNIST inputs. Bottom row: posterior-mean reconstructions.

6.3 Latent representation

Because the learned representation is 20-dimensional, plotting two raw coordinates would be arbitrary. Figure 3 embeds 5,000 posterior means with t-SNE using perplexity 30, PCA initialization, 1,000 iterations, and random seed 42. The well-separated digit clusters indicate that the unsupervised representation captures class-related structure. Local neighborhoods are meaningful, but the global distances and cluster sizes should not be interpreted as metric properties of the original 20-D space. Labels are used only after training for coloring.

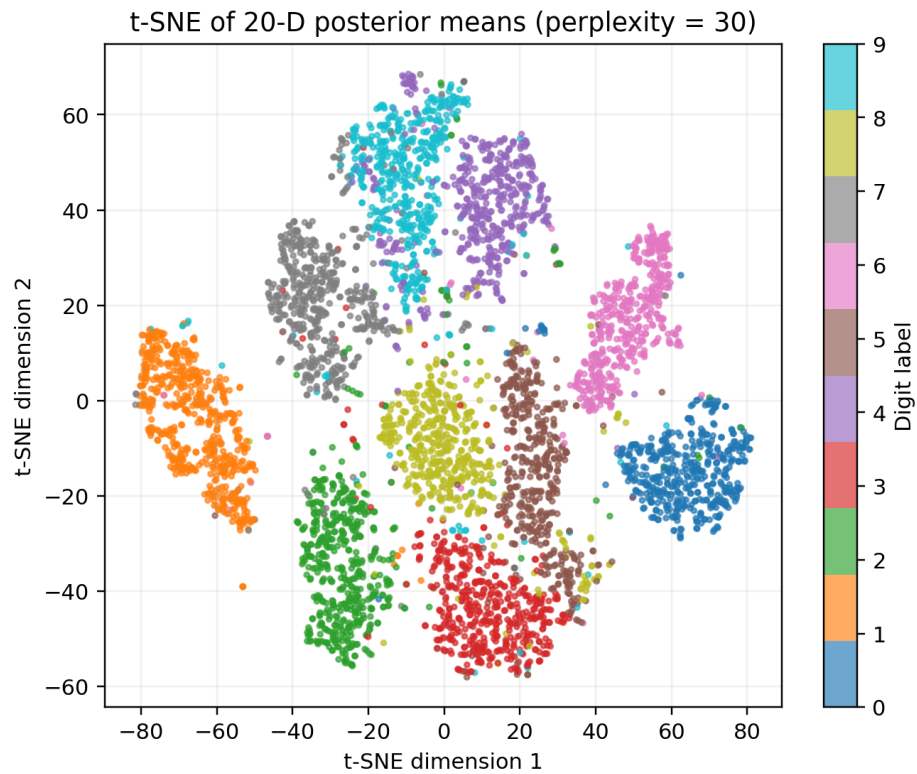


Figure 3: Fixed-seed t-SNE embedding of 5,000 twenty-dimensional posterior means (perplexity 30). Labels are used only for coloring after training.

6.4 Generative behavior

Figure 4 linearly interpolates between posterior means for five pairs of held-out digits. Every row changes gradually from its source to its destination: $0 \rightarrow 1$, $2 \rightarrow 3$, $4 \rightarrow 5$, $6 \rightarrow 7$, and $8 \rightarrow 9$. Smooth intermediate images show that the 20-D latent representation has learned coherent local geometry even though it cannot be displayed directly as a plane. The figure is generated by `infer.py -task interpolate`, which calls `interpolate` and `save_latent_interpolations`.

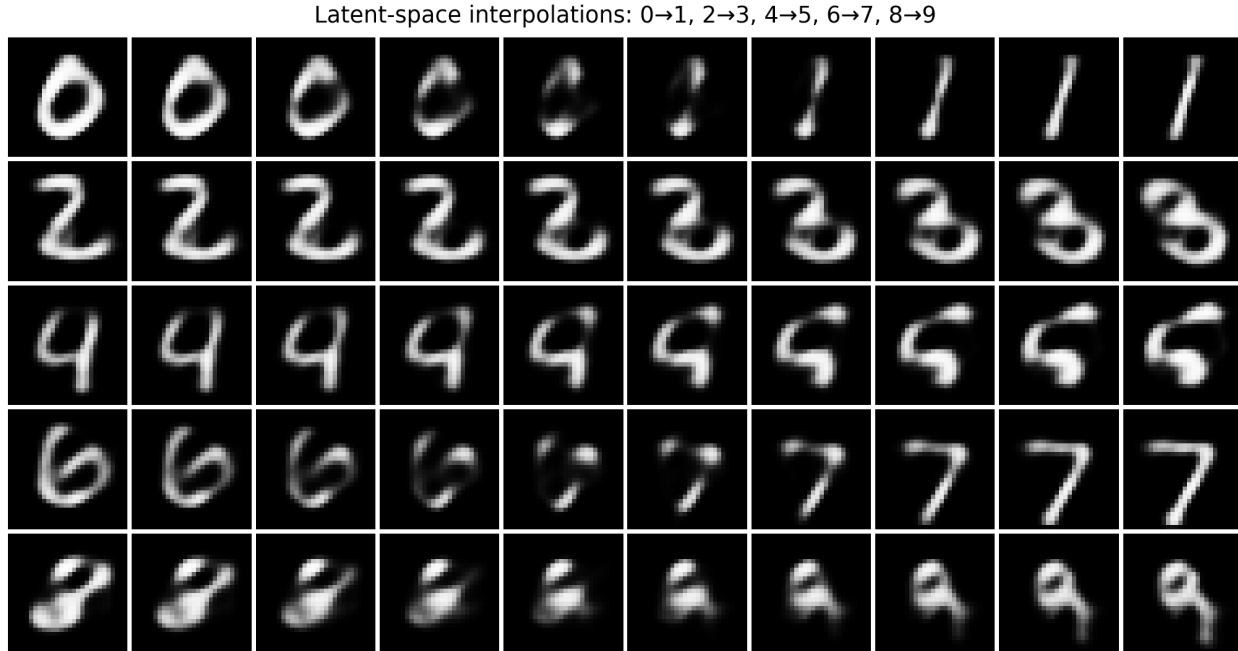


Figure 4: Ten-step posterior-mean interpolations between five pairs of held-out digit classes.

Independent samples $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ in Figure 5 produce substantially more varied digit-like images than the 2-D model. The `generate` task in `infer.py` calls `generate` and `save_prior_samples`. Ambiguous samples remain because the decoder factorizes pixels as independent Bernoulli variables and the run is shorter than the paper’s longest training horizons.



Figure 5: One hundred independent samples from the learned decoder with $z \sim \mathcal{N}(0, I)$.

7 Validation and Interpretation

7.1 Sanity checks

Table 6: Checks used to decide whether the reproduction was successful.

Check	Evidence	Status
Objective improves	Test negative ELBO decreases by 58.85 nats/image from epoch 1 to 50.	Pass
Generalization is stable	Final train-test gap is 2.34 nats/image.	Pass
Posterior is active	Final KL is 24.698 nats/image, not approximately zero.	Pass
Reconstructions retain identity	Posterior-mean outputs preserve strokes, loops, and digit identity.	Pass
Latent structure is meaningful	Fixed-seed t-SNE forms digit-related clusters; interpolations are smooth.	Pass
Prior generates recognizable data	Prior samples are varied and predominantly digit-like.	Pass
Exact paper comparison	No identical training horizon or baseline was available.	Not claimed

7.2 What the experiment demonstrates

The reproduction supports the paper’s central mechanism. A single differentiable objective trains the inference and generative networks jointly. The analytic KL term supplies a principled regularizer, while reparameterization permits ordinary backpropagation through latent samples. The learned prior is not merely a compression device: sampling from it produces new observations, and continuous paths in latent space decode to continuous changes in image space.

The experiment also makes the reconstruction–regularization trade-off visible. The preliminary two-dimensional code was convenient to plot directly but compressed away stroke details. Increasing to $J = 20$ sharply improves reconstruction and sample diversity. Direct visualization is recovered after training with t-SNE for neighborhood structure and explicit latent interpolations for continuity; neither visualization changes the learned model.

8 Conclusion and Limitations

This write-up reproduces the paper’s MNIST VAE with the same probabilistic ingredients and closely matched architectural choices. After 50 epochs, the final 20-D model reaches a held-out negative ELBO of 112.599 nats/image, 53.466 nats/image better than the preliminary two-dimensional model. It reconstructs unseen digits sharply, forms class-related neighborhoods in a fixed-seed t-SNE embedding, supports smooth semantic interpolations, and generates diverse digit-like samples from the standard-normal prior.

The main limitations are deliberate. First, the final study reports one 20-D run and uses the 2-D run only as a diagnostic comparison; it is not a multi-seed latent-size ablation. Second, the wake-sleep and Monte Carlo EM baselines are omitted. Third, training covers 3 million examples, whereas the paper plots substantially longer horizons. Fourth, the paper mentions a small weight-decay term without giving its coefficient, so this reproduction omits it rather than choosing an undocumented value. Fifth, t-SNE preserves local neighborhoods but can distort global distances. Finally, the one-sample ELBO is a lower bound and should not be read as an exact marginal log-likelihood.

A stronger follow-up would train $J \in \{2, 5, 10, 20\}$ under an equal compute budget, report multiple seeds, add importance-weighted likelihood estimates, and measure sample quality with a classifier-based precision/recall analysis. Those extensions are useful scientific comparisons but are not required to validate the AEVB mechanism reproduced here.

A How to Run the Reproduction

From the `vae_mnist_training/` project directory:

```
python3 -m pip install -r requirements.txt
python3 train.py \
  --epochs 50 \
  --batch-size 100 \
  --hidden-dim 500 \
  --latent-dim 20 \
  --learning-rate 0.01 \
  --optimizer adagrad \
  --seed 42 \
  --data-dir ../../work/data \
  --output-dir ../../work/experiment_z20_project
```

After training, load the saved checkpoint and regenerate Figure 2, Figure 4, and Figure 5 without optimization:

```
python3 infer.py \
  --checkpoint pretrained/checkpoint.pt \
  --data-dir ../../work/data \
  --output-dir ../../work/inference_figures \
  --task all \
  --seed 42
```

Replace `all` with `reconstruct`, `interpolate`, or `generate` to produce only Figure 2, Figure 4, or Figure 5, respectively.

Required packages:

```
python3 -m pip install torch==2.9.1 torchvision==0.24.1 \
  numpy matplotlib pillow scikit-learn
```

The output directory contains `checkpoint.pt`, `config.json`, `metrics.json`, `history.csv`, and five PNG figures. On CUDA or MPS hardware the script selects an accelerator automatically; otherwise it runs on CPU. Validate shapes and gradient flow before a full run with:

```
python3 -m unittest tests/test_smoke.py
```

To regenerate figures from the delivered checkpoint, use `render_checkpoint.py` as documented in the project README.

B Loss-to-Code Map

Table 7: Direct correspondence between the derivation and implementation.

Mathematical quantity	PyTorch expression	Note
σ	<code>torch.exp(0.5 * logvar)</code>	Log-variance is numerically convenient.
z	<code>mu + std * randn_like(std)</code>	Equation (7).
$-\log p_{\theta}(x z)$	<code>binary_cross_entropy _with_logits</code>	Sums over pixels and batch.
$D_{\text{KL}}(q_{\phi} \ p)$	<code>-0.5 * sum(1 + logvar - mu**2 - exp(logvar))</code>	Exact diagonal-Gaussian result.
$-\mathcal{L}$	<code>reconstruction + kl</code>	Minimized by the optimizer.

C References

References

- [1] D. P. Kingma and M. Welling. Auto-Encoding Variational Bayes. *arXiv:1312.6114*, 2013.
- [2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

- [3] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [4] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 2019.